

# Procesamiento e Interpolacion de Imagenes Tactiles para un Gripper Robotico utilizando C++ y Python

---

## Sistemas Operativos

### **Autores:**

**Zineb Hamman:** [https://github.com/thecoolerstray/Practica\\_4\\_SO.git](https://github.com/thecoolerstray/Practica_4_SO.git)

**Naela Khaldi :** [https://github.com/naelakh/Practica\\_4\\_SO.git](https://github.com/naelakh/Practica_4_SO.git)

### **1. Explicacion del sistema:**

En esta practica se desarrollo un sistema capaz de procesar información táctil obtenida desde un sensor robotico. El sistema esta compuesto por un cliente desarrollado en C++ y un servidor implementado en Python.

El sistema trabaja con capturas táctiles almacenadas en un archivo JSON. Cada captura contiene una matriz de presión de tamaño 16x16. El objetivo principal es aumentar la resolución de dichas matrices hasta 128x128 mediante interpolación bilineal manual.

El programa desarrollado en C++ realiza la lectura del archivo JSON, valida las dimensiones de cada matriz, aplica la interpolación y posteriormente envía los datos al servidor Python mediante protocolo HTTP.

El servidor desarrollado con Flask recibe las matrices interpoladas y genera automáticamente imágenes táctiles utilizando Matplotlib y NumPy. Además, el sistema completo permite representar visualmente la distribución de presión detectada por el sensor táctil del gripper robótico.

Para el desarrollo se utilizaron las siguientes librerías:

- **En C++ :** nlohmann/json y, libcurl
- **En Python :** Flask , NumPy , Matplotlib

### **2. Explicacion de la interpolacion :**

La interpolación implementada en esta práctica es interpolación bilineal. El objetivo de esta técnica es aumentar la resolución de las matrices originales manteniendo transiciones suaves entre los valores de presión.

La matriz original tiene dimensiones de 16x16 y se transforma en una matriz de 128x128. Para cada nuevo píxel generado, el algoritmo calcula su posición equivalente dentro de la matriz original. Después, se seleccionan los cuatro vecinos más cercanos (**q11, q21, q12, q22**) y se calcula un nuevo valor utilizando interpolación horizontal y vertical.

El proceso utiliza factores de escala para relacionar las posiciones de la nueva matriz con las posiciones de la matriz original:

$$\text{escala} = (16 - 1) / (128 - 1) , \text{gx} = j \times \text{escalaX} , \text{gy} = i \times \text{escalaY}$$

Después, se calculan las distancias fraccionales dx y dy, que permiten combinar los cuatro vecinos más cercanos mediante la siguiente ecuación:

$$\text{Valor} = q11(1-dx)(1-dy) + q21dx(1-dy) + q12(1-dx)dy + q22dx dy$$

La interpolación fue implementada manualmente en C++ utilizando operaciones matemáticas básicas.

### **3. Arquitectura cliente-servidor:**

La arquitectura del sistema está dividida en dos partes principales:

- **Cliente C++ :**

El cliente fue desarrollado en C++ y está compuesto por:

tactile.h / tactile.cpp / main.cpp

Sus funciones principales son:

- leerJSON() : Leer el archivo JSON.
- validarMatriz() : Validar matrices 16x16.
- interpolacionbilineal() : Aplicar interpolación bilineal.
- matriculaJSON() : Convertir matrices a formato JSON.
- enviarPOST(): Enviar datos mediante HTTP POST.

La lectura del archivo JSON se realizó utilizando la librería nlohmann/json, permitiendo convertir fácilmente los datos almacenados en estructuras C++. La validación garantiza que todas las capturas tengan exactamente 16 filas y 16

columnas antes de iniciar el procesamiento. Una vez realizada la interpolación bilineal, las matrices son convertidas nuevamente a formato JSON y enviadas al servidor Python.

El archivo main.cpp coordina todo el flujo de ejecución del sistema, procesando cada captura táctil y mostrando información del proceso en consola.

#### - **Servidor Python:**

El servidor fue desarrollado en Python utilizando Flask y funciona bajo una arquitectura de escucha local en la dirección 127.0.0.1:5000.

Su función principal consiste en recibir las matrices interpoladas enviadas desde el cliente C++ mediante peticiones HTTP POST. Una vez recibidos los datos, el servidor valida las dimensiones de la matriz y convierte la información a estructuras NumPy para facilitar su procesamiento.

Posteriormente, el sistema genera automáticamente imágenes táctiles utilizando Matplotlib mediante la función plt.imshow() y el mapa de color “inferno”, permitiendo representar visualmente la intensidad de presión detectada por el sensor táctil.

Finalmente, las imagenes generadas se almacenan automáticamente en formato PNG con nombres secuenciales como: **capture\_0.png y capture\_1.png**

#### **4. Capturas de resultados:**

El sistema desarrollado logró generar correctamente imágenes táctiles interpoladas a partir de las matrices originales y las imagenes generadas representan mapas de calor donde las zonas de mayor presion aparecen con colores mas intensos gracias al mapa de color “inferno”.

- el arranque del servidor:

```
=====
Servidor Flask iniciado
URL: http://127.0.0.1:5000
Esperando capturas de C++...
=====
```

- El cliente carga el JSON y detecta las 50 capturas:

```
=====
Sistema de procesamiento táctil
=====
Archivo: tactile_captures_50.json
Archivo JSON leído correctamente.
Capturas cargadas: 50
```

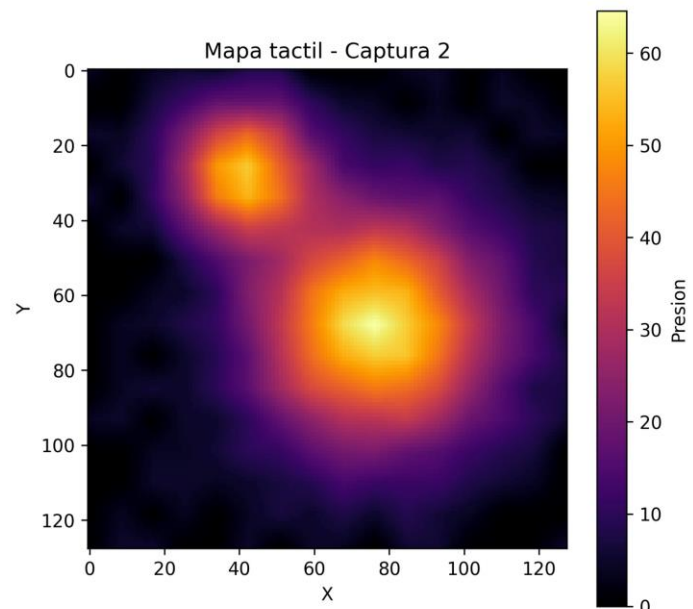
- Procesamiento de una captura táctil:

```
-----
Procesando captura 2
Validacion OK: matriz 16x16
Interpolacion completada: 128x128
Captura 2 enviada correctamente (HTTP 200)
-----
```

- Recepción correcta de la captura interpolada en el servidor Python:

```
=====
Captura recibida: 2
Dimensiones: 128x128
Aviso: captura 2 ya existia, sobrescribiendo.
Imagen guardada: output_images/capture_2.png
=====
127.0.0.1 - - [02/Jun/2026 20:30:09] "POST /capture HTTP/1.1" 200 -
=====
```

- Imagen táctil generada por el servidor Python:



## 5. Problemas encontrados:

Durante el desarrollo de la práctica aparecieron diferentes problemas técnicos :

- **Instalación de librerías y dependencias:** El primer problema estaba relacionado con la instalacion de librerías necesarias tanto en C++ como

en Python. Inicialmente aparecieron errores al instalar dependencias como Flask, NumPy, Matplotlib y libcurl. Estos problemas se solucionaron instalando correctamente las librerías mediante apt y pip3, además de verificar la configuración del compilador y las rutas de inclusión necesarias para libcurl y nlohmann/json.

- **Conversión de datos entre C++ y Python:** Durante la generación de imágenes táctiles surgieron errores relacionados con Matplotlib en entornos sin interfaz gráfica. El problema se solucionó utilizando: `matplotlib.use("Agg")` permitiendo guardar automáticamente las imágenes PNG sin abrir ventanas gráficas.

## 6. Conclusiones:

El sistema desarrollado logró procesar correctamente las 50 capturas táctiles almacenadas en el archivo JSON, validar sus dimensiones y aumentar su resolución desde 16x16 hasta 128x128 mediante interpolación bilineal. Además, se implementó correctamente la comunicación entre el cliente C++ y el servidor Python utilizando HTTP POST, permitiendo transferir las matrices interpoladas para su procesamiento.

La implementación de la interpolación bilineal en el cliente C++ permitió realizar la mayor parte del procesamiento antes del envío de datos al servidor, optimizando el flujo general del sistema y reduciendo la carga de procesamiento en el servidor Flask.

Otro aspecto importante fue la generación de imágenes táctiles utilizando Matplotlib y NumPy. El servidor Flask consiguió recibir correctamente los datos, convertir las matrices a estructuras NumPy y generar mapas táctiles de alta resolución utilizando el mapa de color "inferno".

En conclusión, el sistema desarrollado consiguió procesar correctamente las 50 capturas táctiles, aplicar interpolación bilineal, enviar las matrices mediante HTTP POST y generar mapas táctiles de alta resolución, cumpliendo todos los requisitos establecidos en la práctica.